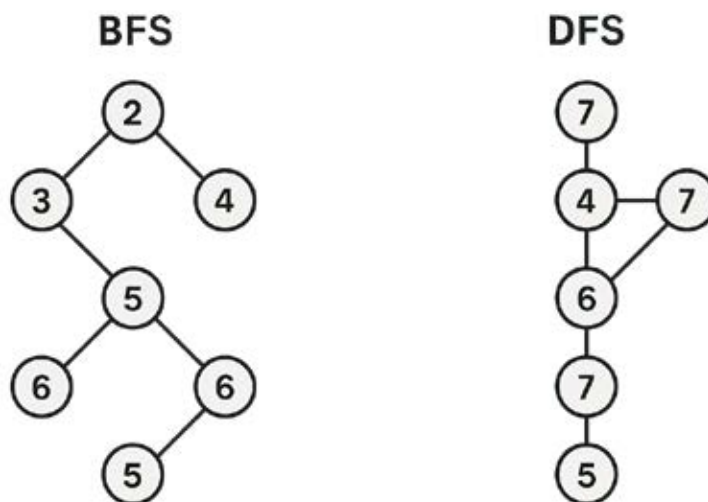


Lisp for Robotics: Implementing Graph Traversal

Created in 1959 by John McCarthy, Lisp is a programming language designed to manipulate symbolic data easily, which is a key characteristic of AI. This language is still used for prototyping and to demonstrate different AI concepts. Here's a short tutorial on how it can help to implement three graph traversal algorithms.



Lisp (LISt Processing) is an excellent solution for high-level robot reasoning, like symbolic computation, rule-based systems, and logic programming. Symbolic AI solutions for planning, knowledge representation, and decision-making are vital for robotics. Lisp is fast in developing and testing these algorithms. Its dynamic typing and interactive Read-Eval-Print environment are a boon to prototype developers. In addition, its powerful functional abstraction capacity makes this language a cutting-edge tool to codify complex robotic operational behaviours into simple step-by-step operations. Lisp can be used for high-level planning of robotic movements, to represent and manipulate a cognitive knowledge base; to interpret controls, logic, inference, and decision trees, and, most importantly, to understand interactive and adaptive behaviours.

Lisp is good for preparing an experimental AI-driven robotic system. This language makes it simple to build knowledge-based agents and cognitive modules for such a system. It can also be used for prototyping of planning or decision-making systems modules. Development of prototypes is fast, and their concise codes make it easy to maintain, debug, and upgrade the algorithms. Then, while

developing real-time low-latency AI-based robotic systems, one can convert these prototype systems to C++ or Python.

Graph traversal algorithms

Pathfinding of a connected graph is an essential function of real-life robotics. There are different strategies, among them graph traversal algorithms, which are easy to understand and are popular for goal finding. Breadth-First Search (BFS), Depth-First Search (DFS), and A* are the algorithms of graph traversal; among them, BFS and DFS are uninformed, and A* is an informed search algorithm. The paths of a robot are represented as a connected graph or tree, and these algorithms are used to explore and search through the graphs and trees. BFS and DFS are examples of uninformed search algorithms because they don't use any knowledge about the goal or the structure of the graph to guide their search. A* is an example of an informed search algorithm because it uses a heuristic (an educated guess) to estimate the distance to the goal, which helps it to prioritise more promising paths.

In essence, BFS, DFS, and A* are fundamental tools used in computer science and artificial intelligence to navigate and explore complex relationships represented in graphs and trees.